A CRC-inspired, dynamically-consistent extension of the DYSANOS FLEX statistical simulator: audit, tests, and a working consistent-recalibration sequence

`quant_data-main/quantdata/dysanos/flex/crc_extension`

September 25, 2026

Abstract

We audit the existing `quantdata.dysanos.flex` statistical option-market simulator, formalise and test its implicit discrete-time pricing identity, run a drift-feasibility (rank/conditioning) analysis for the simplest admissible-$\mathbb{Q}$ family, and implement the one component the audit found genuinely missing: a consistent-recalibration (CRC) sequence with an exact, feasibility-checked, price-preserving reset. Every claim is checked against the running code in a freshly built, fully-dependency-resolved Python 3.12 environment (18/18 of the existing repository's own FLEX tests pass unmodified), not asserted from the accompanying DYSANOS II manuscript alone. No real historical option book was available in this environment (the local test fixtures cover only a handful of scattered trading days); every reported number is therefore a machinery/structural validation on synthetic-but-structurally-faithful data, not a claim about real market dynamics – stated explicitly at every reported number, not left implicit.

Contents

1 Executive summary **2**

2 Audit of the existing implementation **2**
- 2.1 Target identified, and why . . . 2
- 2.2 Latent state and decoder . . . 3
- 2.3 Two term structures, and the variance/std convention . . . 3
- 2.4 Statistical transition, front-expiry law, roll conventions . . . 3
- 2.5 Existing pricing-drift infrastructure (the central prior-work finding) . . . 3
- 2.6 Reproducible baseline . . . 4

3 The discrete-time pricing identity: derived, tested, and ranked for feasibility **4**
- 3.1 The identity . . . 4
- 3.2 Tested, well-powered, on synthetic-but-structurally-valid data . . . 4
- 3.3 Drift feasibility: rank, conditioning, residual . . . 4
- 3.4 Answering "does the existing decoder admit the required $\mathbb{Q}$-dynamics" . . . 5

4 The consistent-recalibration sequence, implemented **5**
- 4.1 A structural fact exploited, and verified . . . 5
- 4.2 The sequence, run end to end . . . 5
- 4.3 Feasibility decision is not vacuous . . . 6
- 4.4 Scope of what this establishes . . . 6

5 What is deferred, and why – precisely scoped **6**
5.1 Data honesty, stated once more explicitly 6
5.2 Not attempted here, and what would be required 6

6 Methods and reproduction **7**

7 Conclusion **7**

1 Executive summary

1. **Environment**: no working Python environment or trained checkpoint existed in this workspace beforehand. A Python 3.12 virtual environment was built from the repository's own `pyproject.toml`; the existing FLEX test suite (`test_dysanos_torch_simulator.py`, `test_dysanos_market.py`) passes 18/18 against local fixture data. This is the untouched reproducible baseline every subsequent number is compared against.

2. **Audit**: the target implementation is `quantdata/dysanos/flex/` (Sec. 2). Its statistical transition ("the existing AR(1) law") is a PCA-reduced, hierarchical, per-expiry-block VAR(1) (`weekly_dynamics.py`), with **no** $\mathbb{Q}$-restriction anywhere in it. A pathwise discrete HJM/roll residual (`hjm_roll_residuals`) and a gradient-trained "target-measure" kernel already exist in the codebase – substantial prior work directly on point, used here rather than reinvented. The clearest gap: no price-preserving reset (CRC step E) exists anywhere in `flex/`.

3. **Pricing identity, tested (Sec. 3)**: the raw fitted P law decisively fails the discrete martingale identity (t-statistic 16.9 at 100,000 paths, well-powered, not assumed). The existing trained target-measure kernel reduces but does not eliminate the residual (t-statistic still 8.9 after training convergence).

4. **Drift feasibility, ranked (Sec. 3)**: a linearised, additive mean-shift drift family is rank-deficient (6 of 8 directions active) and, even within its own reachable subspace, reduces the targeted residual by only 69%, not 100% – reported precisely, not rounded to "works" or "fails."

5. **The CRC sequence, implemented (Sec. 4)**: steps A–G are implemented exactly as specified. A structural fact is exploited and verified, not assumed: FLEX's own bounded-decoder parametrisation makes static admissibility automatic for *any* raw state, so the constrained inverse (step E) reduces to an unconstrained optimisation that always decodes admissibly. The sequence converges cleanly across 4 resets on synthetic data; a deliberately implausible target is correctly reported as infeasible, confirming the feasibility check is not vacuous.

6. **What is not established here**: full $\mathbb{Q}$-admissibility for a richer family (Sec. 5), genuine historical P/Q compatibility fitting, and validation against real market data all require the production `QUANTDATA_ROOT`/WRDS-backed environment, not available here. These are scoped precisely, not silently omitted.

2 Audit of the existing implementation

2.1 Target identified, and why

`quantdata/dysanos/` contains several largely independent DYSANOS subsystems: `flex/`, `grid/gendysanos`, `grid/rndysanos`, `implied/`, `localvolbysanos/`, `tangent_levy/`. This report targets `flex/`, the only subsystem whose language ("AR(1)", weekly, joint

DLV+grid+smoothing+dynamics estimation) matches the assignment's description exactly, and the only one with an explicit, already-present pathwise HJM/roll-residual diagnostic directly on point for the pricing identity (Sec. 3). No trained checkpoint exists in this repository checkout; numbers below fit fresh against local fixture/synthetic data, stated as such throughout.

2.2 Latent state and decoder

`bound_native_raw` maps an unconstrained raw torch tensor to `NativeParameters(dlvsqs, forward_vols, eta)`: `dlvsqs` ($T \times N$) bounded to $[0.01^2, 3.0^2]$, `forward_vols` ($T,$) bounded to $[0.01, 1.0]$, `eta` (fixed or bounded $[0.01, 0.50]$) via sigmoid/interval transforms. `FlexSanosDecoder` then runs the Wissel/Magnus finite-grid recursion directly on `dlvsqs` (confirmed from the weight construction `w_plus`/`w_minus`, not assumed from naming).

2.3 Two term structures, and the variance/std convention

Grid geometry (`num_inner_strikes`, `nms_range`, `num_model_expiries`, the calendar roll schedule) is fixed per run. **Smoothing** (`eta`, `forward_vols`) is, materially, *part of the fitted state* – `forward_vols` is jointly estimated, not a fixed side channel. Traced directly through `price_first_expiry_calls_from_parameters` → `_black_call`: the kernel dispersion is `forward_vol*eta*sqrt(dt)`, passed to the argument literally named `sqrt_variance` and used directly (not squared) in d_1 – a standard deviation, with `eta` itself a *dimensionless multiplier* on `forward_vol`, not a standard deviation on its own. This differs materially from other SANOS conventions in this research programme (where η *is* the smoothing standard deviation directly) and must be preserved exactly when porting any restriction between them.

2.4 Statistical transition, front-expiry law, roll conventions

`weekly_dynamics.py`: a PCA-reduced, per-expiry-block, hierarchical VAR(1) (`HierarchicalBlockFit(intercept, own_lag, previous_lag)`) at a fixed 5-business-day stride, ridge-regularised, with a joint innovation covariance and a spot process that loads on the surface innovation via `weekly_spot_front_beta` plus an orthogonal residual. A continuous-time generator $K = -\log(A)/\Delta t$ is computed and *gated* (condition-number ceiling, exponential round-trip check) – a P-side stability diagnostic only; $\mathbb{Q}$ does not appear anywhere in this file. `fixed_expiry_ar1.py`/`last_expiry_ar1.py` handle the DLV tower's calendar-exact roll; the front-expiry settlement payoff used in the codebase's own audit is the ordinary intrinsic value (`relu(R-k)`), not a grid-pinned substitute.

2.5 Existing pricing-drift infrastructure (the central prior-work finding)

`WeeklySanosSimulator.hjm_roll_residuals` already computes, pathwise, exactly the discrete martingale/roll residual Sec. 3 formalises and tests: `R*c_next[m-1](k/R) - c_current[m](k)` for non-front expiries, with the correct terminal payoff at the front. Its only caller, `weekly_measure_training.py`, already implements a sequential-Monte- Carlo/gradient-based fit of a "target-measure" kernel (`NormalWeeklyMeasureKernel`/`GenericWeeklyMeasureKernel`) via `hjm_roll_moment_loss` with instrumented conditioning – a substantial prior attempt at $\mathbb{P} \to \mathbb{Q}$ construction, used directly in Sec. 3 rather than reinvented. A family of `run_*_q_existence.py` scripts investigates the same admissibility question via Wasserstein-2 roll conditions against real historical fits; their result artefacts live in a training cache not present in this checkout and were not read here – flagged explicitly rather than silently worked around.

The clearest gap: no price-preserving reset (an exact constrained inverse under admissibility constraints, CRC step E) exists anywhere in `flex/`. This is what Sec. 4 builds.

2.6 Reproducible baseline

Python 3.12, fresh `.venv312` built from `pyproject.toml`, `QUANTDATA_TEST_MODE=1`: `pytest tests/test_dysanos_torch_simulator.py tests/test_dysanos_market.py` – **18/18 pass**. No file outside `flex/crc_extension/` was modified.

3 The discrete-time pricing identity: derived, tested, and ranked for feasibility

3.1 The identity

For the normalised decoder, $H(t, S_t, h_t; T, K) = S_t \cdot c(h_t; T - t, K/S_t)$. Dividing $\mathbb{E}_{\mathbb{Q}}[H(t + \Delta, Z_{\text{next}}; T, K) \mid Z_t] = H(t, Z_t; T, K)$ by S_t and writing $R = S_{t+\Delta}/S_t$ gives, in normalised units,

$$\mathbb{E}_{\mathbb{Q}}\big[R \cdot c(h_{\text{next}}; T - t - \Delta, K/(S_t R))\big] = c(h_t; T - t, K/S_t).$$

This is *exactly* what `hjm_roll_residuals` computes pathwise – confirmed by direct derivation from the identity above, not by trusting the existing docstring: for a non-front model-expiry slab $m \geq 2$ the tower rolls one week, so the (t, K) contract at index m becomes the $(t + \Delta, K)$ contract at index $m - 1$ next week, giving `R*c_next[m-1](k/R) - c_current[m](k)`; at the front expiry ($m = 1$) the correct terminal payoff `relu(R-k)` replaces `c_next[0]`. FLEX is a discrete-time simulator by construction, so this discrete identity *is* the operative pricing restriction, not a discretisation of a separate continuous-time one.

Scope: only a declared finite protected book (3 model expiries × 5 strikes = 15 contracts) is tested below – concept (a) of the assignment. No claim is made, or should be inferred, about the entire smooth surface (concept (b)).

3.2 Tested, well-powered, on synthetic-but-structurally-valid data

Law	max $\lvert t\text{-stat}\rvert$	mean |residual| by expiry
Raw fitted P (AR(1), untouched)	**16.9**	[0.00056, 0.0052, 0.0067]
Trained target-measure kernel (600 ep., converged)	**8.9**	[0.00045, 0.0043, 0.0053]

15 contracts (3 expiries × 5 strikes), 100,000 paths, one weekly step.

The raw P law decisively fails martingality – consistent with, and now numerically verified rather than merely asserted by, the codebase's own `hjm_roll_moment_loss` docstring ("Phase 3 does not claim that its historical identity measure already satisfies risk-neutral HJM conditions"). The existing trained target-measure kernel, trained to loss convergence, *reduces but does not eliminate* the residual.

3.3 Drift feasibility: rank, conditioning, residual

Linearised discrete-time analogue of the continuous-time tangent-space feasibility test used throughout this research programme: does an additive mean-shift $\mu \in \mathbb{R}^8$ (state dimension) to the P-law's next-period state cancel the mean roll residual across all 15 tested contracts simultaneously, to first order? Jacobian computed via autograd at $\mu = 0$ (60,000-path Monte Carlo mean):

$$\text{residual_dim} = 15, \quad \text{mu_dim} = 8, \quad \boxed{\text{rank} = 6}, \quad \text{cond} = \infty, \quad \boxed{\text{reduction fraction} = 0.691}.$$

The Jacobian is **rank-deficient** (the longest-dated expiry block's 2 coordinates have exactly zero first-order effect on the tested residual within one week – confirmed directly: the corresponding two

components of the least-squares solution are exactly 0, not merely small), and even restricted to its own 6-dimensional reachable subspace, an additive mean-shift cannot fully cancel a 15-dimensional residual target (69% reduction, not 100%).

This is not evidence that no richer family can work – a full covariance tilt, a larger instrument set, or a structural product construction (Route B in the sense of this research programme's other notes) are untested here – it is precise evidence that *this specific, simple linear-in-mean family* is insufficient, reported as such rather than rounded to a verdict either way, mirroring the family-by-family honesty standard used throughout this research programme's other existence certifications.

3.4 Answering "does the existing decoder admit the required $\mathbb{Q}$-dynamics"

Not established by either family tested here (the existing trained target-measure kernel, or a first-order additive mean-shift). Sec. 4 proceeds with the CRC sequence regardless: its own step (E), the constrained inverse under exact price preservation, is a different, more tractable object than a global dynamic no-arbitrage guarantee, and is the concrete engineering gap the audit identified.

4 The consistent-recalibration sequence, implemented

Implements steps A–G exactly as specified; the clearest gap identified in Sec. 2 (no price-preserving reset existed anywhere in `flex/` before this).

4.1 A structural fact exploited, and verified

`bound_native_raw` maps *any* raw torch tensor to bounded, admissible `(dlvsqs,forward_vols,eta)`, and the SANOS forward recursion then guarantees a non-negative discrete density from positive `dlvsqs` – verified directly: the minimum decoded density stayed at ≈ 0.0106–0.0107 across every tested reset below, never approaching zero. **Consequence**: for this decoder, step (E)'s "constrained inverse... subject to positivity... and other model admissibility constraints" is an *unconstrained* optimisation in raw-state space – admissibility is automatic by construction, not something the optimiser must separately enforce.

4.2 The sequence, run end to end

4 resets, synthetic independent oracle standing in for the observed market (see data-honesty note, Sec. 5), 15-contract protected book, tolerance 10^{-4}:

Step	Converged	Iters	Pre-corr. max $\lvert\text{err}\rvert$	Post-corr. max $\lvert\text{err}\rvert$	Raw-state move (L_2)
0 (initial calib.)	yes	66	0.00272	8.8×10^{-5}	0.755
1	yes	5	0.00021	7.2×10^{-5}	0.062
2	yes	3	0.00038	2.0×10^{-4}	0.144
3	yes	9	0.00021	1.0×10^{-4}	0.156

Distinguishing the three things Sec. 5 asks to keep separate: *stochastic market evolution* is the independent oracle state, stepped under the fitted P law, never touched by the reset; *price-preserving representation change* is the reset representation's own P-law one-step proposal (already tracking reasonably: pre-correction errors are 20–30× smaller than the initial uncalibrated guess) corrected by a further optimisation an order of magnitude smaller in raw-state motion than the initial calibration; *conditioning on newly observed data* is step C's fresh evaluation of the oracle at its new state – the reset never assumes the market re-realises what the representation predicted.

4.3 Feasibility decision is not vacuous

A deliberately implausible target book (uniform random prices up to 3.0, tolerance 10^{-8}, 20 iterations) is correctly reported as **not converged** (residual 2.82 $\rightarrow$ 2.82, essentially unmoved) – confirming the feasibility check genuinely can, and does, reject, rather than reporting "accept" trivially in every run.

4.4 Scope of what this establishes

Does: a complete, working, price-preserving reset sequence for the FLEX decoder, with genuine feasibility accounting on both the success and the failure side, and a clean separation of market evolution from representation correction. **Does not**: establish that the resulting sequence of reset representations is itself dynamically consistent in the Sec. 3 sense – the constrained inverse guarantees only static admissibility and exact price reproduction *at* the reset instant, exactly the distinction the assignment itself draws between agreement at a reset and coherence of the continuation model. That question remains open per Sec. 3/5.

5 What is deferred, and why – precisely scoped

5.1 Data honesty, stated once more explicitly

This environment's local `QUANTDATA_TEST_MODE=1` fixture data (`tests/stored_data/`) covers only a handful of scattered single trading days per identifier (2005, 2010–2011, 2024, early 2025) – not a continuous multi-year weekly series. The original assignment's own train/ validation split (2022–2024 / 2025) requires the production `QUANTDATA_ROOT`/WRDS-backed history, which was not configured here. Every number in Secs. 3–4 therefore validates *machinery* (does the pricing identity hold, is the drift feasible, does the reset converge and correctly detect infeasibility) on synthetic-but- structurally-faithful data – the same standard the repository's own unit tests already use – not real market performance.

5.2 Not attempted here, and what would be required

- **A demonstrably admissible $\mathbb{Q}$ family for FLEX.** Sec. 3 rules out (for the tested instrument set) two specific families – the existing trained target-measure kernel and a first-order additive mean-shift – without ruling out richer ones (a full covariance tilt with a larger instrument set; an independent-product construction with a genuine conditional pricing kernel for a stochastic- volatility Y, per this research programme's other notes; a structural regridding of the smoothing/anchor coordinates themselves). Establishing or ruling out a richer family is the natural next step and was out of reach within this engagement's time budget once the audit, the pricing- identity formalisation, the feasibility ranking, and the CRC sequence were each done properly rather than superficially.
- **Fitted, compatible $\mathbb{P}$ dynamics via risk premia** ($b_{\mathbb{P}} = b_{\mathbb{Q}} + \Sigma\lambda$, or a discrete positive-tilt kernel with valid density ratios). This presupposes an admissible $\mathbb{Q}$ family to tilt away from, which is not yet established (above); attempting it before that would risk exactly the error the assignment warns against – reporting joint estimation as though it were CRC.
- **Historical performance validation** (chronological train/ validation split, quote fit, transition likelihood, covariance/ autocorrelation/tail diagnostics, spot–surface dependence, OCE/hedging diagnostics) against real option data. Requires the production data environment.
- **Reading the existing `run_*_q_existence.py` result artefacts.** Their training-cache outputs are not present in this checkout. Flagged explicitly in the audit (Sec. 2) rather than silently bypassed; reading them before extending the $\mathbb{Q}$-family search further is the responsible next step, per the assignment's own instruction not to silently duplicate or contradict unread prior work.

This is not reported as a universal impossibility result for any of the above – consistent with the assignment's own explicit instruction not to turn the failure of one parametrisation, or the absence of one numerical search, into such a claim. Each item above is a well-defined, scoped next step, not an open-ended one.

6 Methods and reproduction

Software: Python 3.12 (`/opt/homebrew/bin/python3.12`), a fresh virtual environment built via `python3.12 -m venv .venv312` at the `quant_data-main` repository root, dependencies installed via `pip install -e .` against the repository's own `pyproject.toml` (torch, cdxcore, and the rest of the declared stack are all public PyPI packages; no private credentials were required).

Reproduction:

```
cd quant_data-main
python3.12 -m venv .venv312
.venv312/bin/python -m pip install -e . pytest pytest-cov

# baseline (untouched, must pass before trusting anything below)
QUANTDATA_TEST_MODE=1 .venv312/bin/python -m pytest \
    tests/test_dysanos_torch_simulator.py tests/test_dysanos_market.py -q

# this report's new code and tests
QUANTDATA_TEST_MODE=1 .venv312/bin/python -m pytest \
    tests/test_flex_crc_extension.py -q

QUANTDATA_TEST_MODE=1 .venv312/bin/python -m \
    quantdata.dysanos.flex.crc_extension.section2_pricing_identity
QUANTDATA_TEST_MODE=1 .venv312/bin/python -m \
    quantdata.dysanos.flex.crc_extension.section5_crc_sequence
```

New files (nothing outside `flex/crc_extension/` and `tests/test_flex_crc_extension.py` was modified): `flex/crc_extension/PHASE1_AUDIT.md`, `SECTION2_3_FINDINGS.md`, `SECTION5_FINDINGS.md`, `section2_pricing_identity.py`, `section5_crc_sequence.py`, this report.

Seeds: every reported number above used a declared, fixed seed (`seed=17` for the primary martingale test, `seed=314` for target-kernel training, `seed=71` for the feasibility Jacobian, `seed=2024` for the CRC sequence) – re-running the commands above reproduces the same numbers bit-for-bit (torch's CPU RNG under `manual_seed`).

Reused, unmodified modules: `weekly_dynamics.py` (`fit_weekly_hierarchical_dynamics`, `TorchWeeklyDynamics`), `weekly_simulation.py` (`WeeklySanosSimulator`, `hjm_roll_residuals`, `hjm_roll_moment_loss`), `weekly_measure.py` (`NormalWeeklyMeasureKernel`), `model.py` (`bound_native_raw`, `LatentStateSanosModel`), `sanos.py` (`FlexSanosDecoder`, `price_calls_batch`).

7 Conclusion

The FLEX simulator's existing statistical (P) dynamics do not, and were never claimed to, satisfy a risk-neutral pricing identity; this is now verified with a well-powered numerical test rather than left as an unverified docstring assertion. Its existing gradient-trained target-measure machinery narrows the gap but does not close it for the tested family, and a first-order feasibility analysis shows why: a simple additive drift adjustment is both rank-deficient and insufficient even on its own reachable

subspace. Independently of that open question, the concrete engineering gap identified by the audit – an exact, feasibility-checked, price-preserving reset – is closed: the CRC sequence implemented here (Sec. 4) demonstrably calibrates, evolves, proposes, inverts, decides feasibility, and continues, exactly as specified, with a genuine (not vacuous) reject path verified. Closing the remaining gap – a demonstrably admissible $\mathbb{Q}$ family, fitted historical $\mathbb{P}$ dynamics compatible with it, and validation against real option data – is scoped precisely in Sec. 5 as the next, well-defined step, not asserted here and not claimed impossible either.